Clase 2 (Unidad 1) — E/S de datos, formatos y validaciones

Duración total: 4 horas | Proyecto: carga inicial de datos de TaskFlow

Parte 0 — Antes de empezar (10 min)

Abran la carpeta `taskflow/` en VS Code (si no la tenían abierta), abran una terminal nueva y **activen el entorno virtual** (Clase 1, Parte 2.4 Paso 3):

```
source venv/bin/activate
```

(Windows PowerShell: `venv\Scripts\Activate.ps1`). Confirmen que aparece `(venv)` en el prompt antes de seguir. Todo lo que hagamos hoy sigue partiendo de que están parados en la carpeta raíz `taskflow/`.

Parte 1 — Repaso y estructuras de control (35 min)

1.1 Objetivo

Antes de tocar Django, reforzamos en Python puro (usando el intérprete interactivo, `python3`, como en la Clase 1) el manejo de listas de diccionarios y estructuras de control, porque son la base de cualquier importador de datos.

Abran una terminal Python (`python3`, con el `(venv)` activo) y prueben, línea por línea, siguiendo al docente:

```
productos = [
    {"nombre": "Laptop", "precio": 1200, "stock": 15},
    {"nombre": "Mouse", "precio": 25, "stock": 5},
]

for p in productos:
    if p["stock"] < 10:
        print(f"Bajo stock: {p['nombre']}")
```

Repasando pieza por pieza: `productos` es una **lista** (`[...]`) donde cada elemento es un **diccionario** (`{...}`), es decir, pares `clave: valor`. `for p in productos:` recorre la lista elemento por elemento, y en cada vuelta la variable `p` toma el valor de un diccionario distinto. `p["stock"]` accede al valor asociado a la clave `"stock"` de ese diccionario puntual.

> 💡 **Nota del docente**
> Este mismo patrón —recorrer una lista de diccionarios y filtrar con `if`— es exactamente lo que vamos a hacer con las tareas que importemos hoy desde archivos reales.

Parte 2 — Formatos de datos: JSON, CSV y XML (80 min)

2.1 Crear la carpeta para los comandos personalizados de Django

Salgan del intérprete interactivo (`exit()`) si seguía abierto. Django busca comandos personalizados dentro de una estructura de carpetas específica, con nombres exactos. Parados en `taskflow/`:

```
mkdir -p core/management/commands
```

`mkdir -p` crea, en un solo comando, tanto `core/management/` como `core/management/commands/` (el flag `-p` crea las carpetas intermedias que falten). En Windows PowerShell, si `-p` no funciona, créenlas en dos pasos: `mkdir core\management` y luego `mkdir core\management\commands`.

Django, además, requiere un archivo `__init__.py` (vacío) dentro de cada una de esas dos carpetas nuevas, para reconocerlas como **paquetes de Python** importables:

```
touch core/management/__init__.py
```

```
touch core/management/commands/__init__.py
```

(En Windows, si `touch` no existe, créenlos desde VS Code: clic derecho sobre la carpeta → New File → `__init__.py`, dejándolos vacíos).

La estructura final para esta parte debe quedar:

```
taskflow/
  core/
    management/
      __init__.py
      commands/
        __init__.py
```

2.2 Crear una carpeta de datos de ejemplo

También en `taskflow/` (mismo nivel que `manage.py`):

```
mkdir data
```

2.3 JSON

JSON (JavaScript Object Notation) representa datos estructurados como objetos (`{}`) y listas (`[]`), igual que los diccionarios y listas de Python que acabamos de repasar — por eso es tan directo de traducir entre ambos.

Creen el archivo `taskflow/data/tasks_provider_a.json` con este contenido exacto:

```
[
  {"title": "Diseñar wireframes", "priority": "alta", "status": "pendiente"},
  {"title": "Configurar CI", "priority": "media", "status": "pendiente"}
]
```

Para probar cómo se lee, en una terminal Python (`python3`), parados en `taskflow/`:

```
import json

with open("data/tasks_provider_a.json", encoding="utf-8") as f:
    data = json.load(f)

print(data)
```

`with open(...) as f:` abre el archivo y garantiza que se cierre automáticamente al terminar el bloque (aunque ocurra un error en el medio). `json.load(f)` lee el contenido del archivo abierto y lo convierte de texto JSON a estructuras nativas de Python (en este caso, una lista de diccionarios) — de ahí que después `print(data)` muestre exactamente eso.

2.4 CSV

CSV (*Comma-Separated Values*, valores separados por coma) es un formato tabular de texto plano: una fila por línea, columnas separadas por comas, con la primera fila como encabezado.

Creen `taskflow/data/tasks_provider_b.csv`:

```
title,priority,status
Redactar README,baja,pendiente
Revisar PR #12,alta,en_progreso
```

Léanlo con el módulo estándar `csv` (viene incluido con Python, no requiere `pip install`):

```
import csv

with open("data/tasks_provider_b.csv", encoding="utf-8") as f:
    reader = csv.DictReader(f)
```

```
    for row in reader:
        print(row)
```

`csv.DictReader` convierte automáticamente cada fila en un diccionario, usando la primera línea del archivo como nombres de clave (`title`, `priority`, `status`). Sin `DictReader`, tendrían que usar `csv.reader` y acceder a cada columna por posición numérica (`row[0]`, `row[1]`...), mucho menos legible.

2.5 XML

XML (eXtensible Markup Language) representa datos con etiquetas anidadas (`<etiqueta>...</etiqueta>`), un formato más verboso que JSON pero todavía muy usado en sistemas legados y ciertas integraciones empresariales.

Creen `taskflow/data/tasks_provider_c.xml`:

```
<tasks>
  <task>
    <title>Migrar base de datos</title>
    <priority>alta</priority>
    <status>pendiente</status>
  </task>
</tasks>
```

Léanlo con `xml.etree.ElementTree`, también de la librería estándar:

```
import xml.etree.ElementTree as ET

tree = ET.parse("data/tasks_provider_c.xml")
root = tree.getroot()
for task in root.findall("task"):
    print({
        "title": task.find("title").text,
        "priority": task.find("priority").text,
        "status": task.find("status").text,
    })
```

`ET.parse(...)` lee el archivo y arma un árbol de nodos en memoria; `tree.getroot()` devuelve el nodo raíz (`<tasks>`); `root.findall("task")` busca todos los nodos hijos directos llamados `task`; `task.find("title").text` navega al nodo `<title>` dentro de cada `<task>` y obtiene su contenido de texto.

> 💡 **Nota del docente**
> Tres proveedores, tres formatos distintos: esto es intencional y muy realista. En un trabajo real, distintos sistemas legados o distintas áreas de una empresa entregan datos en formatos distintos, y el backend tiene que saber normalizarlos a una única representación interna.

Parte 3 — Modelo Task y comando de importación (90 min)

3.1 Un primer modelo simple

En Django, un **modelo** es una clase de Python que representa una tabla de la base de datos: cada atributo de la clase es una columna. Abran `taskflow/core/models.py` (existe, casi vacío) y agreguen:

```
from django.db import models

class Task(models.Model):
    PRIORITY_CHOICES = [("baja", "Baja"), ("media", "Media"), ("alta", "Alta")]
    STATUS_CHOICES = [
        ("pendiente", "Pendiente"),
        ("en_progreso", "En progreso"),
        ("completada", "Completada"),
    ]
```

```
    title = models.CharField(max_length=200)
    priority = models.CharField(max_length=10, choices=PRIORITY_CHOICES, default="media")
    status = models.CharField(max_length=15, choices=STATUS_CHOICES, default="pendiente")
    source = models.CharField(max_length=50, blank=True)  # de qué proveedor vino
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

`models.Model` es la clase base de Django de la que heredan todos los modelos. `CharField` es un campo de texto corto (requiere `max_length`); `DateTimeField(auto_now_add=True)` guarda automáticamente la fecha y hora de creación, una única vez, sin que nosotros tengamos que asignarla a mano. `choices=` restringe los valores posibles de ese campo (Django los va a validar y mostrar como un desplegable en el panel de administración).

Django todavía no sabe que este modelo existe "en la base de datos real": para eso, generamos y aplicamos una **migración**, parados en `taskflow/`:

```
python manage.py makemigrations core
python manage.py migrate
```

`makemigrations core` analiza los cambios en `core/models.py` desde la última migración y genera un archivo nuevo dentro de `taskflow/core/migrations/` (por ejemplo `0001_initial.py`) con las instrucciones de creación de la tabla. `migrate` aplica esas instrucciones sobre la base de datos real (`taskflow/db.sqlite3`), creando la tabla efectivamente. Son dos pasos separados a propósito: `makemigrations` genera el "plan" y `migrate` lo "ejecuta" — esto permite revisar el archivo generado antes de aplicarlo.

3.2 El management command `import_tasks`

Creen el archivo `taskflow/core/management/commands/import_tasks.py` (dentro de la carpeta que armamos en la Parte 2.1) — el **nombre del archivo** es también el nombre con el que se va a invocar el comando:

```
import csv
import json
import xml.etree.ElementTree as ET
from django.core.management.base import BaseCommand
from core.models import Task


class Command(BaseCommand):
    help = "Importa tareas desde múltiples proveedores (JSON, CSV, XML)"

    def handle(self, *args, **options):
        total = 0
        total += self._import_json("data/tasks_provider_a.json", source="proveedor_a")
        total += self._import_csv("data/tasks_provider_b.csv", source="proveedor_b")
        total += self._import_xml("data/tasks_provider_c.xml", source="proveedor_c")
        self.stdout.write(self.style.SUCCESS(f"Importación finalizada: {total} tareas cargadas"))

    def _import_json(self, path, source):
        count = 0
        try:
            with open(path, encoding="utf-8") as f:
                data = json.load(f)
            for item in data:
                self._crear_task(item, source)
                count += 1
        except (FileNotFoundError, json.JSONDecodeError) as e:
            self.stderr.write(self.style.ERROR(f"Error importando {path}: {e}"))
        return count

    def _import_csv(self, path, source):
        count = 0
        try:
```

```
            with open(path, encoding="utf-8") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    self._crear_task(row, source)
                    count += 1
        except FileNotFoundError as e:
            self.stderr.write(self.style.ERROR(f"Error importando {path}: {e}"))
        return count

    def _import_xml(self, path, source):
        count = 0
        try:
            tree = ET.parse(path)
            for task_el in tree.getroot().findall("task"):
                item = {
                    "title": task_el.find("title").text,
                    "priority": task_el.find("priority").text,
                    "status": task_el.find("status").text,
                }
                self._crear_task(item, source)
                count += 1
        except (FileNotFoundError, ET.ParseError) as e:
            self.stderr.write(self.style.ERROR(f"Error importando {path}: {e}"))
        return count

    def _crear_task(self, item, source):
        if not item.get("title"):
            self.stderr.write(self.style.WARNING("Item sin título, se ignora"))
            return
        Task.objects.create(
            title=item["title"],
            priority=item.get("priority", "media"),
            status=item.get("status", "pendiente"),
            source=source,
        )
```

Comentarios sobre las piezas clave: `BaseCommand` es la clase que Django exige heredar para que algo cuente como un comando de `manage.py`; el método `handle` es el que se ejecuta cuando corremos el comando. Los métodos `_import_json`, `_import_csv`, `_import_xml` empiezan con guion bajo por **convención** (indica "uso interno de esta clase", no forman parte de la interfaz del comando). `Task.objects.create(...)` es la forma en que el **ORM** de Django crea una nueva fila en la tabla `Task`, sin escribir SQL. El bloque `try/except` en cada importador garantiza que si un proveedor falla (archivo faltante, JSON corrupto), los otros dos igual se ejecuten.

3.3 Ejecutar el comando

Parados en `taskflow/`, con el entorno virtual activo:

```
python manage.py import_tasks
```

Deberían ver en la terminal el mensaje `Importación finalizada: 4 tareas cargadas` (2 del JSON + 1 del CSV + 1 del XML).

Para verificarlo visualmente, necesitamos un usuario administrador. Si todavía no lo crearon:

```
python manage.py createsuperuser
```

Les va a pedir usuario, email (opcional) y contraseña (no se ve mientras la escriben, es normal). Luego, en `taskflow/core/admin.py`, registren el modelo:

```
from django.contrib import admin
from .models import Task

admin.site.register(Task)
```

Con el servidor corriendo (`python manage.py runserver`), entren a `http://127.0.0.1:8000/admin/`, inicien sesión con el superusuario, y verifiquen que las 4 tareas estén cargadas con su `source` correspondiente.

Parte 4 — Guardar el avance en Git y GitHub (25 min)

Ya no hace falta `git init` (se hizo una única vez en la Clase 1). El flujo, siempre parados en `taskflow/`:

```
git status
```

Revisen que aparezcan como nuevos: `core/management/`, `core/migrations/0001_initial.py`, `data/`, y los cambios en `core/models.py`, `core/admin.py`. **No debería aparecer** `db.sqlite3` (está en `.gitignore` desde la Clase 1).

```
git add .
git commit -m "Agrega modelo Task y comando de importación multi-proveedor (JSON/CSV/XML)"
git push
```

Noten que a diferencia de la Clase 1, acá no hace falta `git branch -M main` ni `git remote add`: eso ya quedó configurado y no se repite. Alcanza con `add`, `commit` y `push` (repasen el documento **Git y GitHub** si tienen dudas sobre alguno de estos comandos).

Parte 5 — Cierre y tarea

5.1 Resumen

- Manejo de archivos JSON, CSV y XML con librerías estándar de Python.
- Primer modelo Django (`Task`) y su migración (`makemigrations` + `migrate`).
- Un *management command* que integra **múltiples fuentes de datos** con manejo de errores por proveedor (uno que falla no frena a los demás).

5.2 Tarea para la casa

1. Agreguen un cuarto proveedor en formato JSON (`data/tasks_provider_d.json`) con al menos un registro **inválido** (sin `title`) y verifiquen que el comando lo reporte con `WARNING` sin romperse.
2. Agreguen logging con el módulo `logging` de Python en lugar de `self.stderr.write`, guardando los errores en un archivo `import.log` (agréguenlo también a `.gitignore`).
3. Guarden el avance repitiendo el flujo de la Parte 4.

5.3 Próxima clase

Unidad 2: instalaremos y configuraremos **Django REST Framework** en serio, modelaremos `Project` con su relación a `Task`, y expondremos los primeros endpoints de solo lectura.

Diccionario de términos de esta clase

- **CSV (Comma-Separated Values):** formato de texto tabular, con una fila por línea y columnas separadas por comas.
- **Excepción:** un error que ocurre durante la ejecución de un programa y que puede "capturarse" con `try/except` para manejarlo sin que el programa se detenga.
- **Management command:** un comando personalizado, propio del proyecto, ejecutable con `python manage.py <nombre>`.
- **Migración:** un archivo generado por Django que describe cómo crear/modificar las tablas de la base de datos a partir de los modelos.
- **Modelo (model):** una clase de Python que representa una tabla de la base de datos.
- **`__init__.py`:** archivo (habitualmente vacío) que le indica a Python que una carpeta debe tratarse como un paquete importable.

- **Panel de administración (admin):** interfaz web que Django genera automáticamente para gestionar los datos de los modelos registrados.
- **Superusuario:** un usuario de Django con acceso total al panel de administración.
- **XML (eXtensible Markup Language):** formato de texto que representa datos mediante etiquetas anidadas.